

Assignment 5: Voxel Rendering

作者：王洵

1 实验原理

1.1 轴对齐包围盒

轴对齐包围盒 (axis-aligned bounding box, AABB) 用各坐标轴上的最小、最大坐标近似物体所占空间。记

$$\mathbf{b}_{\min} = (x_{\min}, y_{\min}, z_{\min}), \quad \mathbf{b}_{\max} = (x_{\max}, y_{\max}, z_{\max}),$$

则盒内任意点 $\mathbf{p}$ 满足 $x_{\min} \leq p_x \leq x_{\max}$ 等三个不等式。多个子盒取并集后，得到覆盖全部有限几何的更大包围盒。

对仿射变换 M ，将局部盒的八个角点 $\mathbf{c}_k$ 变换为 $\mathbf{c}'_k = M\mathbf{c}_k$ ，再对 $\{\mathbf{c}'_k\}$ 取各轴 min/max，得到世界空间中的保守 AABB。变换后体积可能增大，但不会漏掉与物体相交的空间。无限平面没有有限范围，若参与场景盒的并集，场景盒将在某一方向无限延伸，网格划分随之失去意义，故无限平面不计入场景包围盒。

1.2 均匀体素网格与保守栅格化

均匀体素网格 (uniform voxel grid) 将场景 AABB 沿三轴划分为 $n_x \times n_y \times n_z$ 个单元。第 (i, j, k) 个体素边长为

$$\Delta_x = \frac{x_{\max} - x_{\min}}{n_x}, \quad \Delta_y = \frac{y_{\max} - y_{\min}}{n_y}, \quad \Delta_z = \frac{z_{\max} - z_{\min}}{n_z},$$

中心为

$$\mathbf{v}_{ijk} = \mathbf{b}_{\min} + \left((i + \frac{1}{2})\Delta_x, (j + \frac{1}{2})\Delta_y, (k + \frac{1}{2})\Delta_z \right).$$

体素化 (rasterization) 为每个单元记录可能与之相交的图元。保守策略要求：真实相交的单元必须被标记；额外标记一些不相交的单元并不改变后续可视化或加速的正确性上界。

对球心 $\mathbf{c}_s$ 、半径 r 的球，设体素半对角线

$$h = \frac{1}{2} \sqrt{\Delta_x^2 + \Delta_y^2 + \Delta_z^2}.$$

若体素中心 $\mathbf{v}$ 满足 $|\mathbf{v} - \mathbf{c}_s| \leq r + h$ ，则该体素可能与球相交：体素内任意点到 $\mathbf{v}$ 的距离不超过 h ，故到球心的距离不超过 $|\mathbf{v} - \mathbf{c}_s| + h$ 。对一般图元，先求世界空间 AABB，再与体素 AABB 做分离轴重叠测试：两盒在 x, y, z 三轴投影均重叠时判定为重叠。

1.3 射线—盒子求交 (slab 方法)

射线参数化为 $\mathbf{p}(t) = \mathbf{o} + t\mathbf{d}$ 。对 x 方向的一对平面 $x = x_{\min}$ 、 $x = x_{\max}$ ，

$$t_{x,1} = \frac{x_{\min} - o_x}{d_x}, \quad t_{x,2} = \frac{x_{\max} - o_x}{d_x},$$

取 $t_{x,\text{near}} = \min(t_{x,1}, t_{x,2})$ 、 $t_{x,\text{far}} = \max(t_{x,1}, t_{x,2})$ ； y, z 同理。三对 slab 的公共区间为

$$t_{\text{enter}} = \max(t_{x,\text{near}}, t_{y,\text{near}}, t_{z,\text{near}}), \quad t_{\text{exit}} = \min(t_{x,\text{far}}, t_{y,\text{far}}, t_{z,\text{far}}).$$

当 $t_{\text{enter}} \leq t_{\text{exit}}$ 时射线与盒相交。对整个场景盒只需做一次 slab 求交，即可得到进入网格域的起点，不必对每个体素单独求盒交。

1.4 三维数字微分分析器 (3D DDA)

三维数字微分分析器 (3D DDA) 将射线在网格内的路径化为三轴上的独立一维步进。设当前体素为 (i, j, k) ， d_x 的符号决定下一 x 边界面位置；下一次跨越该面的参数为 $t_{\text{next},x}$ ，步长 $\Delta t_x = \Delta_x / |d_x|$ (y, z 类似)。每步取

$$\min(t_{\text{next},x}, t_{\text{next},y}, t_{\text{next},z})$$

对应的方向前进一格，并将该方向的 t_{next} 增加 Δt ，同时记录刚跨越平面的法线（例如沿 $+x$ 进入时，进入面法线为 $(-1, 0, 0)$ ）。相邻体素共享面，射线路径成为单调单元序列；每步做三次标量比较，复杂度与穿过的体素数成正比。

射线原点在网格外时，以 t_{enter} 为步进起点；原点在网格内时，从当前 t_{min} 开始，直至 t 超过 t_{exit} 结束。

1.5 占用可视化与层次变换

占用可视化将每个占用单元视为实心小立方体。相邻占用体素之间的内界面从外部不可见；当某面邻接空体素或位于网格边界时，该面朝向外外部，可用六方向邻居检测只画外露面。按体素内图元数量映射颜色，低密度与高密度区域形成可辨别的梯度，反映栅格化重叠程度。

层次场景图中，变换矩阵沿树向下累积，子图元收到的世界变换为父链矩阵之积。栅格化与包围盒计算在世界空间进行；对于球体等隐式曲面，将体素中心逆变换到局部空间再做距离测试，使旋转、缩放后的体素包络贴合变换后的几何。

2 程序设计与实现

2.1 总体架构

系统在既有光线追踪、场景配置解析与线性代数 / 图像库的基础上，增加轴对齐包围盒、均匀体素网格、射线步进状态与体素内图元列表。通过网格完成 OpenGL 预览与光线求交，同时网格也作为空间索引。设置好三轴分辨率后，由场景根节点包围盒创建网格，并将有限图元保守插入重叠体素；开启网格可视化时，以主光线对网格做 3D DDA 求交，命中体素时以密度材质完成着色。

```
grid = new Grid(sceneBounds, grid_nx, grid_ny, grid_nz);
sceneGroup->insertIntoGrid(grid, NULL);
```

追踪时根据是否可视化网格来选择求交的对象：

```
if (visualizeGrid && grid != NULL)
    hitSomething = grid->intersect(ray, bestHit, tmin);
else
    hitSomething = group->intersect(ray, bestHit, tmin);
```

2.2 包围盒

有限图元在构造时写入包围盒：球体取 $\mathbf{c} \pm r\mathbf{1}$ ；三角形对三顶点各分量取 min/max；无限平面不设包围盒。物体组在加入子物体时合并其包围盒：

```
BoundingBox *childBox = obj->getBoundingBox();
if (childBox == NULL)
    return;
if (bbox == NULL)
    bbox = new BoundingBox(childBox->getMin(), childBox->getMax());
else
    bbox->Extend(childBox);
```

变换节点对一般子物体变换局部盒八角点后取新 AABB；对三角形子物体，作特殊化处理，即将三顶点变到世界空间之后再取包围盒。

2.3 体素网格与栅格化

网格复制场景 AABB，按分辨率计算 $\Delta_x, \Delta_y, \Delta_z$ ，每个单元用可动态增长的列表存放图元指针。默认栅格化将经累积变换后的图元包围盒映射到索引范围，与体素 AABB 重叠则插入。

```
void Object3D::insertIntoGrid(Grid *g, Matrix *m) {
    if (g == NULL || bbox == NULL)
        return;
    g->insertObjectInBBox(bbox, this, m);
}
```

球体以测试半径 $r + h$ 做保守距离判定；存在变换时先将体素中心逆变换到局部空间：

```
float testRadius = radius + halfDiag;
Vec3f voxelCenter = g->getVoxelCenter(i, j, k);
if (m != NULL)
    inv.Transform(voxelCenter);
if ((voxelCenter - center).Length() <= testRadius)
    g->insertObject(i, j, k, this);
```

物体组递归栅格化子物体。变换节点将父链与本地矩阵合成为 $M_{\text{combined}} = M_{\text{parent}} \cdot M_{\text{local}}$ 后下传：

```
Matrix combined = matrix;
if (m != NULL)
    combined = (*m) * matrix;
object->insertIntoGrid(g, &combined);
```

2.4 3D DDA 求交

步进状态保存当前参数 t 、体素索引 (i, j, k) 、三轴 t_{next} 与 Δt 、方向符号及进入面法线。初始化时用 slab 求 $t_{\text{enter}}, t_{\text{exit}}$ ，由入口点确定起始体素并设置三轴 DDA 量：

```
if (inside) {
    tEnter = tmin;
    tExit = tExitBox;
} else {
    if (!hitsBox || tEnterBox > tExitBox)
        return;
    tEnter = tEnterBox;
    tExit = tExitBox;
}
computeVoxelIndex(startPoint, ...);
initAxis(dir.x(), origin.x(), bbMin.x(), dx, i, tEnter, signX, dTx, tNextX);
// y、z 轴同理
```

单步前进取最小 t_{next} ，更新索引与进入面法线：

```
void MarchingInfo::nextCell() {
    if (t_next_x < t_next_y) {
        if (t_next_x < t_next_z) {
            i += sign_x;
            tmin = t_next_x;
            t_next_x += d_tx;
            normal = Vec3f(-(float)sign_x, 0.0f, 0.0f);
        } else { /* 沿 z 前进 */ }
    } else { /* 沿 y 或 z 前进 */ }
}
```

求交沿射线步进，命中第一个占用体素时写入进入面法线与按图元数量分级的密度材质：

```
initializeRayMarch(mi, r, tmin);
while (mi.getTMin() <= mi.getTExit()) {
    int count = getObjectCount(mi.getI(), mi.getJ(), mi.getK());
    if (count > 0 && mi.getTMin() >= tmin && mi.getTMin() < h.getT()) {
        h.set(mi.getTMin(), getDensityMaterial(count), mi.getNormal(), r);
        return true;
    }
    mi.nextCell();
}
```

2.5 OpenGL 预览与交互调试

预览遍历占用体素，当某面邻接空体素或位于网格边界时绘制该面，颜色随重叠图元数由白经绿、青、蓝、黄、橙过渡至红：

```
if (i == 0 || getObjectCount(i - 1, j, k) == 0)
    paintVoxelFace(..., Vec3f(-1, 0, 0), count);
```

主程序解析网格分辨率与可视化开关。交互式画布在可视化模式下绘制全部占用体素，或绘制单条射线所遍历的体素与进入面。按 t 键追踪当前鼠标像素对应射线；按 g 键在「全部占用体素 / 射线遍历体素 / 射线进入面」三种显示间切换，从而对照 DDA 步进顺序。

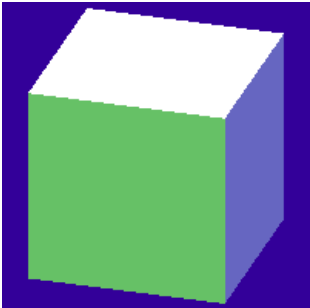
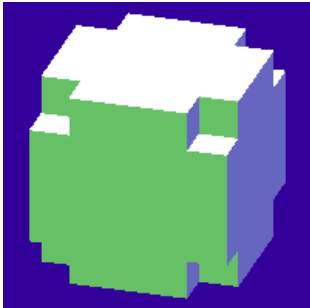
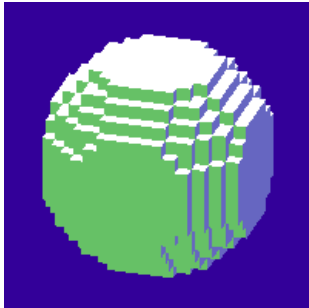
3 实验结果

测试命令形如：

```
# 单球场景; -grid 体素分辨率, -visualize_grid 以网格占用替代场景几何
raytracer -input <单球场景> -size 200 200 -output output5_01a.tga -grid 1 1 1 -visualize_grid
# 多球场景
raytracer -input <多球场景> -size 200 200 -output output5_02a.tga -grid 15 15 15 -visualize_grid
# 约四万面 Stanford bunny
raytracer -input <bunny_40k 场景> -size 200 200 -output output5_09.tga -grid 40 40 33 -visualize_grid
```

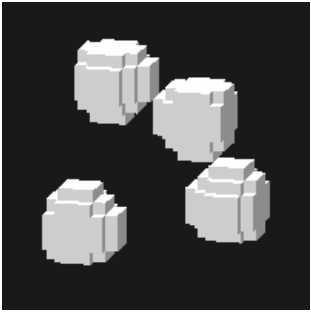
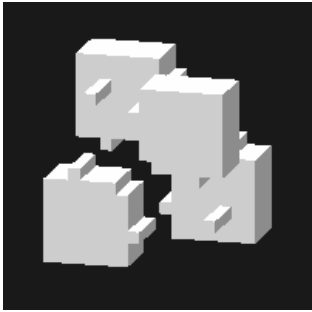
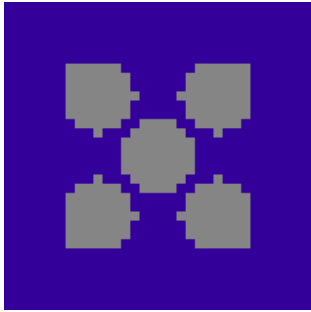
十五组主场景均在 200 × 200、开启网格可视化下渲染。分辨率升高时，占用区域由块状变为更贴近原几何的体素包络；重叠处颜色沿密度梯度加深。

3.1 单球与网格分辨率

1 × 1 × 1	5 × 5 × 5	15 × 15 × 15
		
整场景盒化为单个体素	球体呈粗阶梯状外壳	轮廓更圆滑

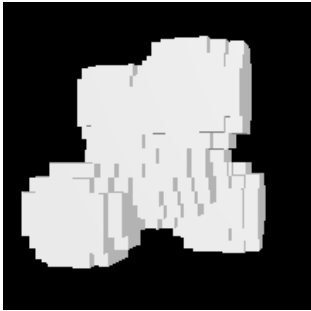
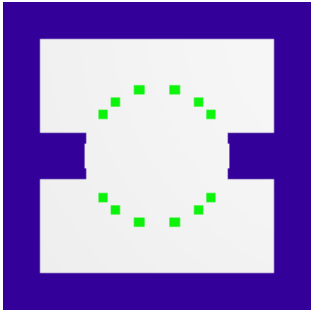
1 × 1 × 1 将整个场景盒化为单个体素；5 × 5 × 5 与 15 × 15 × 15 下球体呈阶梯状外壳，分辨率越高轮廓越圆滑，说明球体距离阈值 $r + h$ 给出了合理的保守占用。

3.2 多球与偏心球

多球 15 × 15 × 15	多球 扁平 15 × 15 × 3	偏心球 20 × 20 × 20
		
重叠区密度色更深	z 向体素粗，侧面呈层状	包围盒不含原点，占用仍贴合球

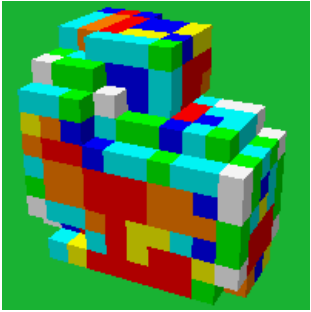
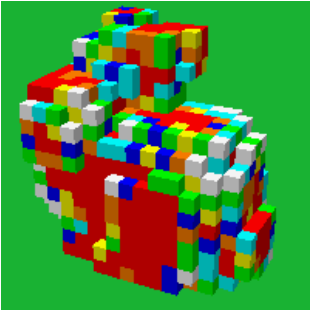
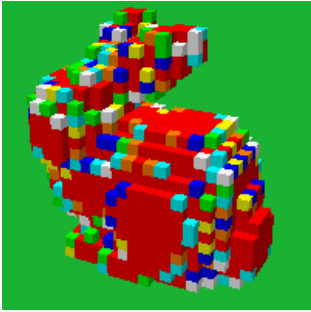
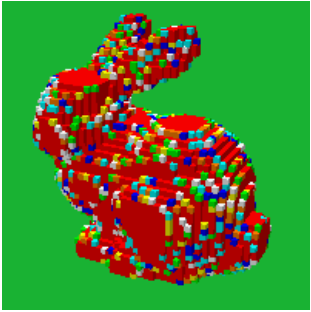
多球重叠区呈更高密度色。扁平网格在 z 方向体素较粗，侧面呈层状。偏心球场景的包围盒不含原点，栅格化结果仍贴合球体位置。

3.3 平面与混合图元

含无限平面 15 × 15 × 15	球体 + 三角形 20 × 20 × 10
	
无限平面不计入场景盒	球面与三角面片占据不同体素

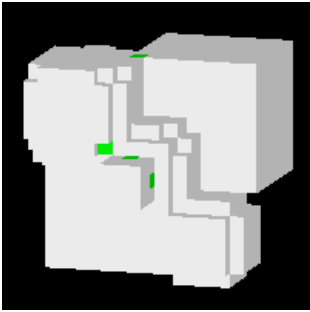
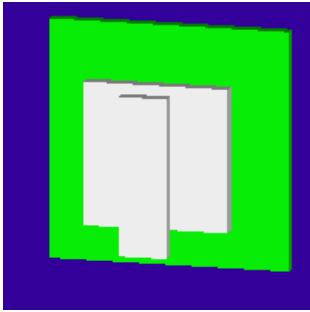
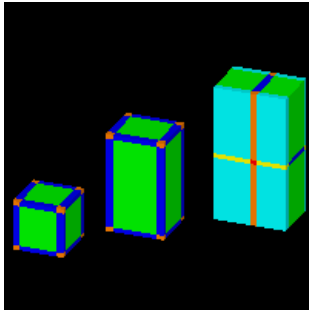
无限平面不参与场景盒计算，网格包围有限图元。混合场景中球面与三角面片占据不同体素区域，密度图区分了两者的栅格化范围。

3.4 Stanford bunny 网格

约 200 面 10 × 10 × 7	约 1k 面	约 5k 面	约 40k 面 40 × 40 × 33
			
粗体素包络	轮廓更清晰	高密度区偏暖	贴近真实网格外形

面数增多时，单个体素内图元数上升，高密度区颜色偏暖；轮廓随三角形密度更接近真实网格外形，说明 AABB 栅格化对大规模三角网格仍适用。

3.5 仿射变换场景

缩放平移 15 × 15 × 15	旋转三角形 15 × 15 × 9	嵌套变换 30 × 30 × 30
		
世界空间 AABB 贴合变换后几何	三顶点取紧 AABB，占用更贴合	矩阵级联后包络仍正确

矩阵级联后的世界空间 AABB 与球体局部距离测试使体素包络贴合变换后几何。旋转三角形对变换后三顶点取紧 AABB，占用范围更贴合物体，与八角点变换方法的结果形成对比。

4 问题记录

旋转三角形场景中，位于场景包围盒 z 上边界的绿色三角形在网格可视化中整块消失。调试占用切片时发现对应层为空，进一步打印插入过程发现了问题。体素索引范围没有对索引作完整的截断，当图元恰落在 $z = z_{\max}$ 时， k_0 被算成 n_z ，再与已被压到 $n_z - 1$ 的 k_1 比较后循环永不执行，因而插入零个体素。补充截断后，该三角形出现在最外层体素中。

扁平网格多球场景中，样例应有的层间长条形体素突起缺失，占用数较样例偏少。原因是球体栅格化的搜索范围与判定半径不一致：距离测试使用保守半径 $r + h$ (h 为体素半对角线)，但三重循环的索引范围却按照几何 AABB ($\mathbf{c} \pm r$) 裁剪。在 z 向体素较粗时， h 主要由 Δ_z 贡献，相邻层体素中心到球心的距离可满足 $|\mathbf{v} - \mathbf{c}| \leq r + h$ ，却从未进入循环。将索引范围在几何盒外再按 h 各向扩展一圈后，成功使占用数增加，中间层出现与样例一致的条状延伸，问题解决。

变换节点对所有子物体一律变换局部 AABB 的八角点后取世界盒。三角形的局部盒本身已大于三角面，旋转后世界盒更松，栅格化多占空体素。对三角形子物体改为将三顶点变换到世界空间后再取 min/max，得到更紧的包围盒，让旋转三角形与嵌套变换场景的占用轮廓更贴合几何。

5 总结

均匀体素网格将连续三维场景离散为轴对齐单元，通过保守 AABB 栅格化与 3D DDA 射线步进，在 OpenGL 预览与光线追踪两条路径上验证占用标记与遍历顺序。球体距离阈值、密度渐变色与外露绘制使体素化质量与重叠程度可直接观察；变换矩阵沿场景树累积后，世界空间栅格化为后续用网格加速射线—图元求交打下基础。

附录 (TODO list 记录)

- ☒ 为球体、三角形等有限图元计算轴对齐包围盒；物体组合并子盒；变换节点对八角点或三角形三顶点取世界空间包围盒
- ☒ 实现轴对齐均匀体素网格，按场景包围盒与三轴分辨率划分单元，并用动态列表存储各体素内图元
- ☒ 实现图元向网格的栅格化
- ☒ 在光线追踪器中由场景包围盒创建网格并插入全部图元，解析网格分辨率与可视化开关
- ☒ 实现网格占用的 OpenGL 预可视化，绘制外露面并按图元数量密度着色
- ☒ 实现 3D DDA 步进状态，以及射线步进初始化与单步前进
- ☒ 用 3D DDA 实现网格与射线求交，命中占用体素时返回进入面法线；可视化求交着色
- ☒ 集成射线树调试：记录命中体素与进入面；按 t / g 键切换全部占用、遍历体素与进入面显示
- ☒ 将三角形等图元按世界空间包围盒栅格化，并以重叠图元数量做密度可视化
- ☒ 变换节点级联矩阵后下传；包围盒八角点变换到世界空间；球体在变换下逆变换体素中心后栅格化